Due Date: 11/4/2026

Assignment 01: Combinational Logic

As a reminder about assignments, they will not be checked off. However, it is highly recommended that you complete them! They are designed in such a way that you'll have an easier time doing the labs—some problems here will be hands-on practice, and others will be "pre-labs" that will be used in the labs. Solutions will be posted shortly after the due date so that you can check your answers—and please do so!

1. Logic Gates

Create SystemVerilog modules for each of the logic gates: NOT, AND, OR, XOR, NAND, NOR. Then, create a module to XNOR all the bits in a 64-bit vector `a` (*hint: it would help to use a reduction operator here*). Here's the skeleton code:

```
module parity(
       input logic [63:0] a,
       output logic parity_result
);

       // FILL THIS IN

endmodule
```

2. K-Maps

(If you are familiar with K-maps already, you may skip past this explanation to the problem.)

A	B	C	F
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

How do we use a truth table to derive a Boolean expression? There are two ways to do so. The first way is by inspecting the output column of the table and finding all the 1s.

A	B	C	F
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

We then inspect the input side. Every time we see a row where F = 1, we will AND together all the inputs in that same row. Once we've done this, we will OR all these groups of inputs, or "products," together. Here, we'd have:

$$F = \overline{A}\overline{B}\overline{C} + \overline{A}\overline{B}C + A\overline{B}\overline{C}$$

This notation for expressing F is called **sum of products (SOP)** notation, because we are ORing (+) groups of ANDs ($\cdot$) together.

Another way to drive a Boolean expression using a truth table is to look for all the 0s on the output side.

A	B	C	F
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

Now, for every row where we see F = 0, instead of ANDing together all the inputs, we will *OR* together all the *complements* of the inputs. Then, we will AND together the groups of ORed inputs (or "sums"). For this example, it will look like this:

$$F = (A+\overline{B}+C)\,(A+\overline{B}+\overline{C})\,(\overline{A}+B+\overline{C})\,(\overline{A}+\overline{B}+C)\,(\overline{A}+\overline{B}+\overline{C})$$

It logically follows that this notation is called **product of sums (POS)** form.

Believe it or not, the SOP and POS forms of F we derived are equivalent! You can see this for yourself if you try simplifying both. There's also more to the terminology. Another term for the product or AND of all inputs in a row is **minterm**, denoted by m_i where i is the row (starting at 0). The sum or OR of all inputs in a row is called a **maxterm**, denoted by M_i. SOP form can therefore also be written using summation notation:

$$F = \Sigma(m_0, m_1, m_4)$$

POS form can be written using product notation:

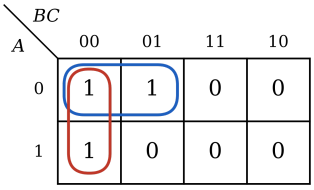
$$F = \prod(M_2, M_3, M_5, M_6, M_7)$$

Now you might be thinking, "How do we choose which form to use?" It's merely a matter of convenience! In this example, you'll notice that we have more rows where F = 1 than rows where F = 0. That means, if we use SOP form, we'll have fewer minterms in our equation than maxterms in the equivalent POS equation—and that's easier for us!

From here, we can go one step further in simplifying these equations. We'll use a nifty diagram called a **Karnaugh map**, or **K-map**. The cells in a K-map contain the outputs for all combinations of inputs, listed above and to the left of the K-map. We can fill in a K-map using the truth table, and here is the K-map for our example:

A \ BC	00	01	11	10
0	1	1	0	0
1	1	0	0	0

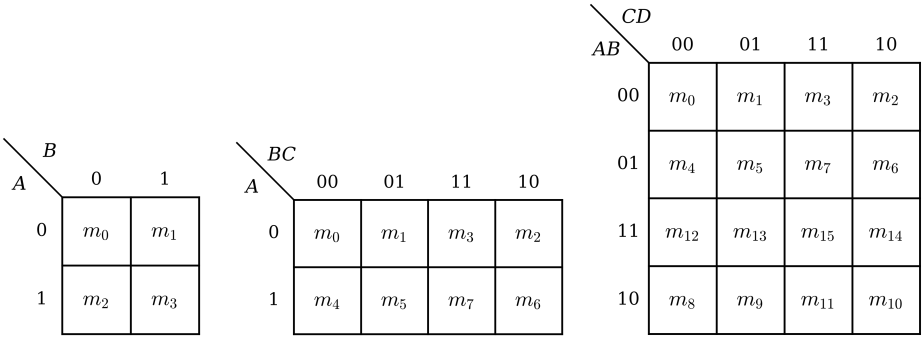
We then circle groupings of 1s. These groupings must either be squares or lines; they cannot span diagonally across the map. Additionally, the K-map is a torus. You can wrap groupings around the sides.



We can now use these groups to find commonalities between the cells. The blue group shares $\overline{A}$ and $\overline{B}$; the red group shares $\overline{B}$ and $\overline{C}$. So, the simplified equation we can make from the K-map is:

$$F = \overline{A}\,\overline{B} + \overline{B}\,\overline{C}$$

The general K-map format for two, three, and four variables using minterms is as follows:



Familiarity with K-maps may be incredibly helpful for this problem...

Given the following truth table, derive both the SOP and POS forms of the Boolean expression. Draw the K-map and use it to simplify the expression. Use the simplified expression to draw a logic gate circuit (*hint: you should end up with four gates*). Finally, create a combinational module in SystemVerilog to model this (*hint: your solution doesn't need to take up more than one line inside the module*).

A	B	C	F
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

3. 4-Bit Comparator

Below is the skeleton for a comparator, that compares 4-bit signals a and b and sets three flags to be 1 or 0. For example, if a > b, gt = 1 and eq = lt = 0. However, the input signals are split up into individual signals. It may help to use the concatenation operator.

If we have three signals `msb`, `mid`, and `lsb`, we can combine them into a three-bit number like so: `{msb, mid, lsb}`. Knowing this, fill in the code!

```
module comparator4bit(
       input logic a3, a2, a1, a0,  // Inputs a and b are split up for some
reason...
       input logic b3, b2, b1, b0,  // Let's combine them using concatenation!
       output logic eq, gt, lt
);

       // FILL THIS IN

endmodule
```

4. Sign Extension

Can you figure out how to do concatenation without manually writing repetitions of the MSB?

```
module sign_extension(
       input logic [6:0] a,
       output logic [15:0] y
);

       // FILL THIS IN

endmodule
```

5. MUX-tryoshka Doll

One way to create a large MUX is by chaining together a bunch of small MUXes. Can you figure out how to create a 4:1 MUX using three 2:1 MUXes? Draw the circuit and write the corresponding SystemVerilog module—create a 2:1 MUX module called `mux2`, and instantiate it several times in `mux4`. Here's something for you to get started:

```
module mux2(
       input logic [1:0] d0, d1,
       input logic s,
       output logic [1:0] y
);

       // FILL THIS IN

endmodule
```

```
module mux4(
       input logic [3:0] d0, d1, d2, d3,
       input logic [1:0] s,
       output logic [3:0] y
);

       // FILL THIS IN: what internal signals might you need to connect the MUXes?

       mux2 lowmux(/* FILL THIS IN */);
       mux2 highmux(/* FILL THIS IN */);
       mux2 finalmux(/* FILL THIS IN */);

endmodule
```

6. Emergency Request Handler

A building's control system has four request lines, `a[3:0]`, which can be active at the same time:

Bit	Source	Priority
`a[3]`	Fire alarm	Highest
`a[2]`	Security breach	
`a[1]`	Power fault	
`a[0]`	Maintenance request	Lowest

Only one request can be served at a time. Design a circuit that outputs a 4-bit one-hot signal `y[3:0]` indicating which request is granted:

- `y` has a single 1 in the position of the highest-priority active request.
- If no requests are active, `y = 4'b0000`.

For example, if the fire alarm and the power fault are both active (`a = 4'b1010`), the fire alarm wins, so `y = 4'b1000`.

Fill in a condensed truth table for this circuit using as few rows as possible. Use X for any input whose value doesn't affect the output (*hint: once the fire alarm is active, does it matter what the other three lines are doing?*).

Implement the circuit in SystemVerilog as a module named `priority_casez`, with `input logic [3:0] a` and `output logic [3:0] y`. Your case statement should have one item per row of your truth table, not one per input combination. Beware: a regular case statement compares every bit exactly, so a 4-bit input needs up to 16 case items. `casez` lets you mark bits as don't-cares with `?`, so one case item can match many input values. This is the code version of an X in a condensed truth table. `?` means "this bit can be anything." For example, `4'b00??` matches any value whose top two bits are 00. Items are checked top to bottom, and the first match wins. If an input matches more than one item, only the first one runs. The order of your items therefore sets their priority. Include a `default` for inputs that match no item, so the output is always assigned.

7. Adders

(If you are familiar with adders, you may skip ahead to the actual problem at the end.)

i. Half Adder

One of the main applications of digital electronics is doing math. One such building block of digital hardware is a half adder. A half adder adds two 1-bit inputs, a and b, and produces a sum bit s and a carry bit c. You can think of it as one column of long addition in base 2. The only interesting case is 1 + 1 = 10 in binary; the column writes down 0 and carries 1 into the next column. If we were to turn this into a truth table, it'd look like this:

a	b	c (carry)	s (sum)
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

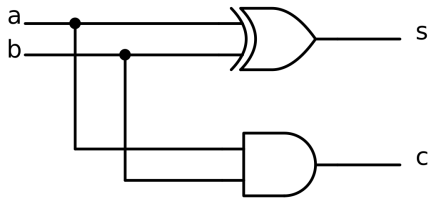
Reading the columns straight off the truth table:

- s is 1 when exactly one input is 1, which is XOR.
- c is 1 only when both inputs are 1, which is AND.

So, a half adder is just one XOR gate and one AND gate.

$$s = a \oplus b$$

$$c = a \cdot b$$



Why is this just a "half" adder? Every column of a multi-bit addition except the rightmost one may receive a carry from the column to its right. However, a half adder has nowhere to put that carry.

ii. Full Adder

So, a half adder is only used for the LSB. A full adder adds three bits: a, b, and a carry-in c_{in} from the column to its right. It produces a sum s and a carry-out c_{out}. The largest possible total is 1 + 1 + 1 = 3 = 11 in binary, so two output bits are always enough.

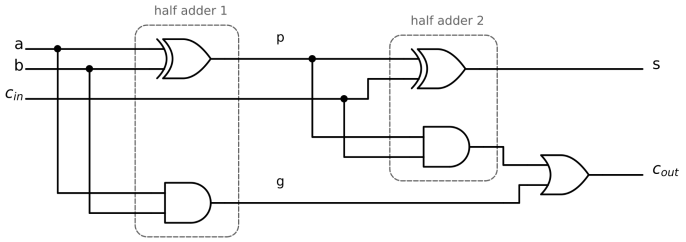
a	b	c_{in}	c_{out}	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

If we keep an eye out for patterns, we may notice that the sum is 1 when an odd number of inputs are 1, and the carry-out is 1 when at least two inputs are 1.

$$s = a \oplus b \oplus c_{in}$$

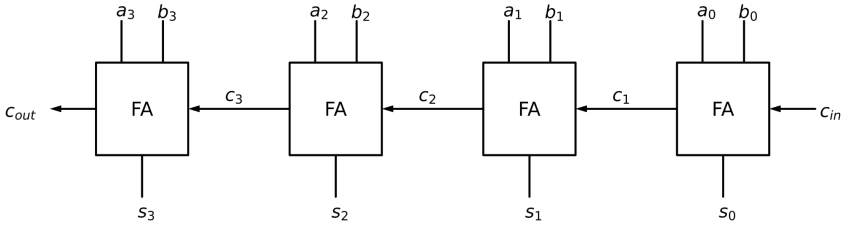
$$c_{out} = a \cdot b + a \cdot c_{in} + b \cdot c_{in}$$

You can get the same result by chaining two half adders and an OR gate!



iii. Ripple Carry Adder

To add two N-bit numbers, line up N full adders and connect each one's c_{out} to the next one's c_{in}. Bit i adds a_i, b_i, and c_i, and passes c_{i+1} to bit i+1. The c_{in} of bit 0 is the adder's overall carry-in (0 for plain addition), and the c_{out} of the top bit is the overall carry-out.



The problem is *speed*. Bit i can't produce its final sum until its carry-in has settled, and that carry depends on bit i−1, which depends on bit i−2, and so on. In the worst case, a carry is created at bit 0 and has to ripple through every stage. Adding 0111 1111 + 0000 0001 is an example: the carry from bit 0 flips every bit on its way to the top.

The delay of a ripple-carry adder therefore grows linearly with its width—a 64-bit ripple adder is roughly 8 times slower than an 8-bit one. So, engineers have designed faster adders aimed at getting around waiting for that pesky carry chain.

iv. Generate and Propagate

We'll create two new signals, generate (g) and propagate (p) that split each column's carry logic into two parts: a) what the column does on its own, and b) what it does with an incoming carry. Both depend only on a_i and b_i, so all of them are ready after one gate delay, before any carry has arrived!

$$g_i = a_i \cdot b_i$$

$$p_i = a_i \oplus b_i$$

a_i	b_i	g_i	p_i	What happens to this column's carry in?
0	0	0	0	**Kills it**: $c_{i+1} = 0$ no matter what
0	1	0	1	**Propagates it:** $c_{i+1} = c_i$

1	0	0	1	**Propagates it:** $c_{i+1} = c_i$
1	1	1	0	**Generates one**: $c_{i+1} = 1$ no matter what

With these two signals, the full adder equations become:

$$c_{i+1} = g_i + p_i \cdot c_i$$
$$s_i = p_i \oplus c_i$$

This doesn't make a ripple-carry adder faster by itself, since c_i still has to arrive. What it does is separate the slow part (the carry chain) from the fast part (g and p), which is the starting point for every fast adder, including the carry-select adder below.

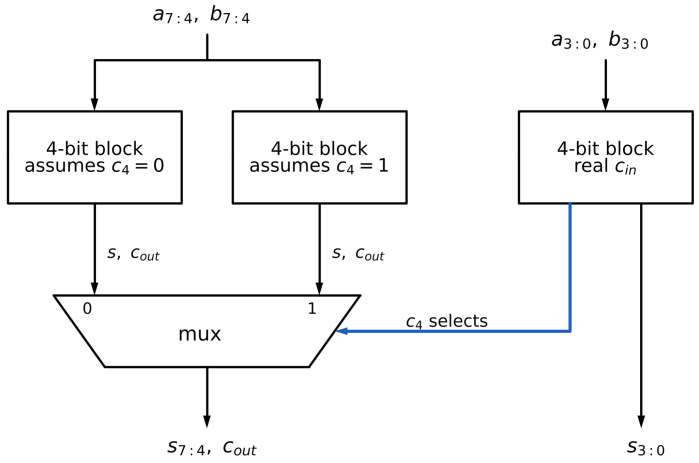
v. Carry-Select Adder

A carry-select adder avoids waiting for the carry by computing the upper bits both ways in advance and choosing the right answer once the real carry arrives.

We split an 8-bit adder into a lower block (bits 3:0) and an upper block (bits 7:4). The upper block only needs one thing from the lower block: the carry c_4. That carry can only be 0 or 1, so the upper block computes *two* complete results *at the same time* as the lower block works. Conceptually, this may be tricky to think about since our brains work sequentially. However, hardware can work *in parallel* if you design it cleverly. Three things happen at once:

- the [3:0] bits compute
- the [7:4] bits compute, assuming a carry-in of 0
- another copy of the [7:4] bits compute, assuming a carry-in of 1

When this stage is done, the lower block has finished and the real c_4 is known. A 2:1 MUX selects the correct upper result using the real value of the carry-in. Choosing between two ready answers takes one mux delay, which is far less than rippling a carry through four more bits!



The cost is hardware: the upper block's carry logic is built twice, and every upper sum bit plus the carry-out needs a mux. That penalty—more area for less delay—is the primary tradeoff of fast adder design.

vi. 8-Bit Carry Select Adder

Build an 8-bit carry-select adder from two 4-bit blocks, using only generate and propagate signals for the carry logic and NO ADDITION OPERATOR. Each 4-bit block takes a[3:0] and b[3:0] and produces two results:

- s0 and c0: the 4-bit sum and carry-out assuming the block's carry-in is 0
- s1 and c1: the 4-bit sum and carry-out assuming the block's carry-in is 1

Inside each block, use only these equations:

$$g_i = a_i \cdot b_i$$
$$p_i = a_i \oplus b_i$$
$$c_{i+1} = g_i + p_i \cdot c_i$$
$$s_i = p_i \oplus c_i$$

Here's the skeleton code:

```
module csel_block4 (
    input logic [3:0] a, b,
    output logic [3:0] s0, s1,   // Sums assuming carry-in = 0 / 1
    output logic c0, c1                // Carry-outs assuming carry-in = 0 / 1
);

      // FILL THIS IN

endmodule



module csel_adder8 (
    input logic [7:0] a, b,
    input logic cin,
    output logic [7:0] s,
    output logic cout
);

      // FILL THIS IN

endmodule
```

Consider the following. A 16-bit carry-select adder is built from four 4-bit blocks in the same way. How many carry stages and how many muxes lie on its critical path? Compare that with a 16-bit ripple-carry adder.